FECAP - FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação
6º Semestre - Turma 6º CCOMP

SMASH

RELATÓRIO DE STATUS E MITIGAÇÃO DE RISCOS

Entrega 2 - Redes de Computadores e Cibersegurança

Projeto 7 - Grupo Smash

Felipe Vallim Soares - 24026060
Guilhermy Mariano Lisboa Garcia - 23025371
Gustavo Oliveira Demetrio - 24026213
Saulo Pereira de Jesus - 24026095

Prof. Victor Bruno Alexander Rosetti

São Paulo
2026

IDENTIFICAÇÃO DO DOCUMENTO

Quadro 1 - Identificação do relatório.

Campo	Informação
Projeto	Smash - parede instrumentada de treino de tênis (Projeto 7)
Disciplina	Redes de Computadores e Cibersegurança - Entrega 2
Data	06/11/2026
Versão do documento	2.0 (a versão 1.0 é o Plano da Entrega 1)
Documento anterior	Plano de Gerenciamento de Riscos, versão 1.0 (Entrega 1, 25/09/2026)
Repositório	*https://github.com/2026-2-NCC6/Projeto7*
Código de referência	Tag *entrega2-ciberseguranca-final*, commit *9d5e68e5ac535063e72ecd3f08426066c5d1a899*, pasta *src/Entrega1*. A Entrega 2 continuou o código da Entrega 1, na mesma pasta; a tag marca o estado entregue agora, e os caminhos citados são relativos a essa pasta
Código-fonte	Acompanha este relatório. Também pode ser baixado em *https://github.com/2026-2-NCC6/Projeto7/archive/refs/tags/entrega2-ciberseguranca-final.zip*

SUMÁRIO

1 INTRODUÇÃO....3
2 METODOLOGIA DA VERIFICAÇÃO....4
3 RESUMO DO STATUS DOS RISCOS....5
4 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO....6
5 EVIDÊNCIAS DE TESTES DE SEGURANÇA....13
6 RISCOS MATERIALIZADOS E NOVOS RISCOS....14
7 COBERTURA DOS REQUISITOS DE SEGURANÇA....16
8 RISCO RESIDUAL E PRÓXIMOS PASSOS....18
APROVAÇÃO....19
ANEXO A - REGISTRO DE RISCOS ATUALIZADO....20
ANEXO B - SAÍDAS DOS TESTES....23

1 INTRODUÇÃO

Este relatório complementa o Plano de Gerenciamento de Riscos entregue na Entrega 1. Naquele documento, a equipe levantou 19 riscos no código e no projeto do Smash e definiu, para cada um, uma resposta e um prazo. A maior parte tinha prazo na Entrega 2 e estava com status "Não iniciado". Este relatório mostra o que foi corrigido no código, onde está cada correção, como ela foi testada e o que ainda falta.

A Entrega 2 da disciplina pede a entrega complementar dos relatórios e as mitigações implementadas no código do sistema. Por isso, este relatório usa a mesma numeração da Entrega 1 (R01 a R19): cada risco aparece com o que mudou desde então, e os riscos novos começam em R20.

O Plano da Entrega 1 assumiu três compromissos para esta revisão: implementar e verificar no código as respostas aos riscos Extremos (R01, R02 e R03), associar cada resposta concluída ao commit que a implementa e repetir a verificação de dependências com npm audit. As respostas aos três riscos Extremos estão implementadas e testadas no código, mas ainda dependem de um último passo fora dele: publicar a API (R01) e gravar o firmware novo na placa (R02 e R03). Os outros dois compromissos foram cumpridos e estão nas seções 4 e 5 e nos Anexos A e B.

1.1 O que mudou no projeto desde a Entrega 1

O hardware mudou durante o desenvolvimento, e algumas descrições do Plano da Entrega 1 ficaram desatualizadas. O Quadro 2 registra as mudanças que afetam a leitura dos riscos.

Quadro 2 - Mudanças no projeto entre as entregas.

Aspecto	Entrega 1	Entrega 2
Parede	Nove alvos em matriz 3 × 3	Seis alvos em grade 3 × 2
Leitura dos piezos	Multiplexador de canais	Cada piezo no seu próprio pino ADC1 do ESP32
Integração MQTT	Prevista (R13)	Fora do escopo. O enunciado cita MQTT só como uma das opções de comunicação; o Smash usa WebSocket entre parede e aplicativo e HTTP entre aplicativo e API
Atuador	Não havia	LED da placa (GPIO 2), acionado pela parede e pelo aplicativo
Registro da sessão	O aplicativo enviava o resultado pronto	O servidor abre a sessão antes do jogo e aceita um único resultado por sessão (R04)

2 METODOLOGIA DA VERIFICAÇÃO

Na Entrega 1, os riscos foram levantados por revisão estática do código. Nesta entrega, cada correção foi verificada por uma evidência que pode ser repetida, como o próprio Plano exigia: a eficácia de uma resposta é medida por commit, configuração ou teste, e não pela intenção declarada. Também foi feita uma nova rodada de busca de vulnerabilidades. Foram usadas seis formas de verificação:

- **Testes automatizados:** cada correção veio com testes de unidade. O backend tem 86 testes e o aplicativo tem 117, e todos passam no commit de referência.
- **Script de testes de segurança:** *npm run security:test* faz contra a API e a parede os ataques de um cliente modificado, como senha errada repetida, partida forjada e conexão sem segredo. Foi rodado no emulador da parede e também pode ser apontado para a placa; a saída está no Anexo B.
- **Busca de segredos no histórico:** o gitleaks varreu todos os commits do repositório, e o histórico foi revisado também à procura de senhas escritas no código.
- **Análise estática do código (SAST):** o Semgrep rodou regras de segurança para TypeScript, Node.js, C/C++, JWT e segredos sobre o backend, o aplicativo, o firmware e os scripts.
- **Varredura da API em execução (DAST):** o OWASP ZAP examinou a API rodando localmente, primeiro no modo baseline e depois em todas as rotas, autenticado e com ataques ativos, a partir da descrição OpenAPI *scripts/zap-openapi.yaml*.
- **Verificação de dependências:** *npm audit --omit=dev* no backend e no aplicativo, antes e depois das correções. O firmware também foi compilado com o arduino-cli para a placa ESP32 Dev Module.

A severidade residual de cada risco usa as mesmas escalas de probabilidade e impacto e a mesma matriz da Entrega 1 (seções 6 e 7 daquele documento). Cinco riscos estão com a correção concluída no código, mas dependem de um passo fora dele. Para todos eles vale o mesmo critério: a severidade residual indicada é a que fica depois desse passo, e até lá vale a severidade da Entrega 1.

Dois passos ainda não foram feitos: gravar o firmware novo na placa ESP32 e publicar a API com certificado reconhecido (seção 8). Por isso R01, R02, R03, R05 e R07 estão como "Concluído no código", com o passo que falta indicado.

3 RESUMO DO STATUS DOS RISCOS

O Quadro 3 resume a situação dos 19 riscos da Entrega 1. Na Entrega 1 havia 13 riscos Extremos ou Altos. Deles, 5 estão concluídos (R04, R06, R09, R11 e R17); 5 estão com a correção pronta no código e só falta publicar a API ou gravar a placa (R01, R02, R03, R05 e R07); 1 foi encerrado (R13, porque o MQTT saiu do projeto); e 2 continuam em andamento fora do código (R14, que é físico, e R15, reformulado para o prazo da apresentação).

Quadro 3 - Situação dos riscos da Entrega 1.

Status	Riscos	Total
Concluído	R04, R06, R08, R09, R10, R11, R12 e R17	8
Concluído no código; falta publicar a API ou gravar a placa	R01 (publicar) e R02, R03, R05 e R07 (gravar)	5
Encerrado	R13 (risco eliminado) e R19 (concluído desde a Entrega 1)	2
Em andamento, fora do código	R14, R15 (reformulado), R16 e R18	4

O Quadro 4 mostra, na matriz da Entrega 1, a severidade residual, já com os riscos novos (seção 6).

Quadro 4 - Matriz de probabilidade e impacto com a severidade residual.

Probabilidade \ Impacto	Muito Baixo	Baixo	Médio	Alto	Muito Alto
Muito Alta	Médio	Alto	Alto	Extremo	Extremo
Alta	Baixo	Médio	Alto	Alto	Extremo
Média	Baixo	**Médio** R21, R25, R27	**Médio** R04, R16, R22, R28	**Alto** R14	Alto
Baixa	Muito Baixo	**Baixo** R12	**Baixo** R05, R06, R10, R11, R20, R26	**Médio** R02, R07, R15, R17, R18	**Médio** R01, R09
Muito Baixa	Muito Baixo	**Muito Baixo** R23	**Baixo** R08, R19	**Baixo** R03	**Médio** R24

- **Extremo:** nenhum (eram 3). **Alto:** 1 (eram 10), o R14, que é físico e não se resolve no código.
- **Médio:** 15 (R01, R02, R04, R07, R09, R15, R16, R17, R18, R21, R22, R24, R25, R27 e R28). **Baixo e Muito Baixo:** 11 (R03, R05, R06, R08, R10, R11, R12, R19, R20, R23 e R26).

4 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO

Esta seção descreve cada correção em quatro partes: como era antes, o que mudou, como foi testado e o que ainda falta. O título de cada item mostra a severidade na Entrega 1 e a severidade residual. O commit e os arquivos de cada correção estão no Quadro 13, no Anexo A.

4.1 R01 - A API usava HTTP sem criptografia (Extremo → Médio, falta publicar)

- **Antes:** o aplicativo enviava e-mail, senha e o token de acesso em texto claro (*http://*) para a API, que escutava em todas as interfaces de rede da máquina.
- **O que mudou:** a API passou a servir HTTPS quando recebe um certificado (variáveis *TLS_KEY_PATH* e *TLS_CERT_PATH*). Com certificado, ela não responde mais por HTTP. Fora do modo de desenvolvimento, o aplicativo não inicia se o endereço da API não começar com *https://*.

  Em desenvolvimento, o HTTP continua permitido, para o aplicativo falar com o notebook da equipe na rede de testes.
- **Como testamos:** com um certificado de teste, a API respondeu por HTTPS e recusou a conexão por HTTP na mesma porta (Anexo B). Cinco testes de unidade cobrem a regra do aplicativo.
- **O que falta:** publicar a API em um servidor com certificado reconhecido, por exemplo atrás de um proxy HTTPS. Até a publicação, vale a severidade da Entrega 1.

4.2 R02 - O WebSocket da parede aceitava qualquer cliente (Extremo → Médio, falta gravar)

- **Antes:** qualquer aparelho conectado ao Wi-Fi da parede podia abrir o WebSocket, receber todos os impactos e mandar comandos.
- **O que mudou:** a parede agora exige um segredo de pareamento. O aplicativo conecta em *ws://<parede>:81/?token=<segredo>*; com o segredo errado ou ausente, a parede responde "nao autorizado" e fecha a conexão.

  A comparação do segredo não para no primeiro caractere diferente, para o tempo de resposta não dar pistas sobre o segredo. Impactos e batimentos vão só para o aplicativo pareado, e não mais para todos os conectados.
- **Como testamos:** testes R02 do Anexo B.1, no emulador da parede; o firmware com a mesma regra compila para o ESP32.
- **O que falta:** gravar o firmware na placa e repetir o teste nela. O canal continua sem criptografia (*ws://*), porque o ESP32 não serve *wss://* com folga, e o segredo

vai no próprio endereço da conexão (R26). O que protege o segredo é a rede: a parede cria a própria rede Wi-Fi com WPA2 e senha forte (R03).

4.3 R03 - A senha do Wi-Fi da parede era previsível e pública (Extremo → Baixo, falta gravar)

- **Antes:** a senha da rede da parede ("smash1234") estava escrita em *config.h*, um arquivo versionado em repositório público.
- **O que mudou:** a senha do Wi-Fi e o segredo de pareamento saíram do *config.h* e foram para o *secrets.h*, que está no *.gitignore* e nunca vai para o Git. O repositório só tem o modelo *secrets_example.h*, com valores de exemplo.

  Sem o *secrets.h*, o firmware não compila e mostra como criá-lo. O README do firmware orienta a gerar senha e segredo fortes para cada parede (*openssl rand -base64 18* e *openssl rand -hex 16*) e a trocar os dois antes de cada apresentação.
- **Como testamos:** sem o *secrets.h*, a compilação falha com a mensagem "Falta secrets.h" (Anexo B); com ele, o firmware compila. No código atual, a senha antiga só aparece no aviso do README que proíbe usá-la.
- **O que falta:** a senha antiga continua no histórico do Git, de onde não pode ser apagada, e continua gravada na placa física. Por isso ela é tratada como vazada: a placa precisa ser gravada com o firmware novo e uma senha nova antes da próxima demonstração. Até lá, vale a severidade da Entrega 1.

4.4 R04 - O resultado da partida era aceito sem prova (Alto → Médio)

- **Antes:** o aplicativo enviava o resultado pronto (nível, pontos, acertos e duração), e o backend aceitava qualquer valor dentro de limites largos. Um cliente modificado podia inventar partidas e subir no ranking.
- **O que mudou:** agora o servidor abre a sessão antes do jogo. Ao começar uma partida, o aplicativo chama *POST /me/sessions/start*. O servidor confere se o nível está liberado para o jogador (no máximo um acima do último concluído) e devolve um ticket assinado com o jogador, o modo, o nível e a hora de início.

  O resultado só é aceito com esse ticket. O servidor recusa o resultado quando o ticket é de outro jogador ou foi alterado, quando o modo ou o nível não batem com os do início, ou quando a duração informada é maior que o tempo real desde o início.

  Também recusa quando há mais batidas do que a parede consegue registrar nesse tempo (uma a cada 200 ms, o intervalo mínimo do firmware) e quando a pontuação passa do máximo possível para aquele número de acertos.

Cada ticket vale para um único resultado. Isso é garantido por uma restrição UNIQUE no banco, que funciona mesmo com duas requisições chegando ao mesmo tempo.

- **Como testamos:** testes R04 do Anexo B.1, na API real, mais testes de unidade no backend e no aplicativo.
- **O que falta:** dentro desses limites, um cliente modificado ainda consegue declarar um resultado plausível que não aconteceu. Provar a partida de ponta a ponta exigiria que a própria parede assinasse os impactos, o que fica como melhoria futura.

4.5 R05 - Qualquer cliente trocava a sessão da parede (Alto → Baixo, falta gravar)

- **Antes:** qualquer cliente conectado podia mandar o comando *session* e trocar o identificador que a parede coloca em cada impacto.
- **O que mudou:** a parede atende um aplicativo por vez. O primeiro que pareia com o segredo certo fica com a parede até sair; um segundo aplicativo recebe "parede ocupada" e é desconectado, mesmo com o segredo certo.

  Só o aplicativo pareado manda comandos. Quando ele sai, a parede volta ao identificador da inicialização e fica livre para o próximo.
- **Como testamos:** testes R05 do Anexo B.1, no emulador da parede.
- **O que falta:** gravar o firmware na placa e repetir o teste nela.

4.6 R06 - Login e cadastro sem limite de tentativas (Alto → Baixo)

- **Antes:** era possível tentar senhas sem limite em */auth/sign-in* e criar contas sem limite em */auth/sign-up*.
- **O que mudou:** dois contadores protegem o login e o cadastro. Por endereço IP: 10 tentativas por minuto, com bloqueio de 5 minutos. Por conta (e-mail): 5 tentativas a cada 15 minutos, com bloqueio de 15 minutos.

  Como o bloqueio dura mais que a janela de contagem, quem insiste fica bloqueado por mais tempo. Acima do limite, a API responde 429 com a mensagem "Muitas tentativas. Aguarde alguns minutos e tente de novo.", e a tentativa fica no log de auditoria (R12), sem a senha.
- **Como testamos:** teste R06 do Anexo B.1: cinco logins errados receberam 401, e o sexto e o sétimo, 429.
- **O que falta:** o bloqueio por conta tem um efeito colateral conhecido: quem souber o e-mail de outra pessoa consegue bloqueá-la por 15 minutos errando a senha de propósito. Esse efeito foi registrado e aceito como R25.

4.7 R07 - A parede não limitava clientes nem comandos (Alto → Médio, falta gravar)

- **Antes:** a parede aceitava vários clientes, mensagens de qualquer tamanho e comandos sem limite de frequência. Um cliente que sumia sem fechar a conexão continuava ocupando a parede.
- **O que mudou:** um aplicativo por vez (R05); comandos com mais de 256 bytes são recusados; acima de 20 comandos por segundo, os excedentes são recusados sem ser interpretados.

  O ping do WebSocket, a cada 5 segundos, desconecta o aplicativo que deixa de responder a 2 pings seguidos.
- **Como testamos:** testes R07 do Anexo B.1, no emulador da parede.
- **O que falta:** gravar o firmware na placa e repetir o teste nela. Inundar o rádio Wi-Fi ainda pode atrapalhar a parede; isso não se resolve em software e fica aceito.

4.8 R08 - CORS aberto e API em todas as interfaces (Médio → Baixo)

- **Antes:** a API aceitava requisições de navegador de qualquer origem e escutava em todas as interfaces de rede (*0.0.0.0*).
- **O que mudou:** o CORS fica desligado por padrão, porque o aplicativo mobile não precisa dele, e só é ligado para as origens listadas em *CORS_ORIGINS*. A interface de escuta virou configuração (*HOST*), com padrão *127.0.0.1*, que só aceita conexões da própria máquina.

  Para a rede de testes, o *.env.example* usa *HOST=0.0.0.0* de forma explícita.
- **Como testamos:** teste R08 do Anexo B.1, mais testes de unidade dos valores padrão.

4.9 R09 - Senha padrão do banco como reserva (Alto → Médio)

- **Antes:** sem a variável *DB_PASSWORD*, a API tentava conectar ao banco com um usuário e uma senha conhecidos.
- **O que mudou:** as variáveis *DB_USER* e *DB_PASSWORD* não têm mais valor padrão no código. Sem elas, a API não sobe e avisa qual variável está faltando.
- **Como testamos:** testes de unidade conferem que a API recusa iniciar sem cada uma das duas variáveis.
- **O que falta:** o ambiente publicado precisa de uma conta de banco própria, com senha forte e só as permissões que a aplicação usa (privilégio mínimo).

4.10 R10 - Token de sete dias e sem revogação (Médio → Baixo)

- **Antes:** o token de acesso valia sete dias, e sair da conta não o invalidava: uma cópia do token continuava funcionando.

- **O que mudou:** cada usuário tem uma versão de token no banco, e o token leva a versão em que foi emitido. Ao sair da conta, o aplicativo chama *POST /auth/sign-out*, a versão sobe e todos os tokens anteriores daquele usuário param de funcionar, nesse aparelho e em qualquer outro.

  A validade padrão caiu de 7 dias para 12 horas. O token de acesso também ganhou um público próprio (audience), para que nenhum outro token assinado com o mesmo segredo, como o ticket do R04, seja aceito no lugar dele.

  Não foi preciso criar a renovação de token prevista no Plano: o aplicativo guarda o token só na memória, e o jogador entra de novo a cada vez que abre o app.
- **Como testamos:** teste R10 do Anexo B.1, mais testes de unidade da revogação.

4.11 R11 - Dependências com alertas conhecidos (Alto → Baixo)

- **Antes:** na Entrega 1, o npm audit mostrava 9 alertas no backend e 18 no aplicativo. Até esta revisão, o backend tinha chegado a 11 alertas, um deles crítico.
- **O que mudou:** o backend foi atualizado do NestJS 10 para o 11, e a biblioteca multer, que continuava com um alerta na versão usada pelo NestJS, foi fixada na versão corrigida (2.5.0). No aplicativo, o *npm audit fix* aplicou as atualizações compatíveis.

  O backend ficou sem nenhum alerta. O aplicativo ficou com 5 alertas altos, todos dentro do *@expo/cli*, a ferramenta de linha de comando do Expo, que roda só no computador do desenvolvedor: o node-forge e uma cópia antiga do zod (3.25.76). O zod que o próprio aplicativo usa é o 4.6.7, já corrigido. Essa ferramenta não entra no aplicativo instalado no celular, então esses alertas foram justificados como não alcançáveis, como o Plano previa.
- **Como testamos:** o *npm audit* foi repetido antes e depois (Quadro 5 e Anexo B.6). O pacote Android gerado com *expo export* foi conferido e não contém a biblioteca node-forge, uma das que têm alerta. As suítes de teste do backend e do aplicativo passaram depois das atualizações.

4.12 R12 - Pouca trilha de auditoria (Médio → Baixo)

- **Antes:** eventos de segurança quase não eram registrados, e a parede só escrevia no monitor Serial.
- **O que mudou:** a API escreve uma linha em JSON para cada login, cadastro e logout, com sucesso ou falha, e para cada requisição recusada (400, 401, 403, 409 e 429) ou com erro do servidor. A linha tem o evento, o resultado, o status, o método, a rota, o IP e o usuário, quando houver.

O e-mail aparece mascarado (*a***@dominio*), e nunca entram senha, token ou o corpo da requisição. Na parede, as recusas de pareamento e os comandos recusados são registrados no Serial.

- **Como testamos:** um login com senha errada e um acesso com token forjado geraram linhas no log, e a senha usada no teste não apareceu em nenhuma delas (Anexo B). Testes de unidade cobrem o conteúdo das linhas.
- **O que falta:** os logs saem no console. Guardá-los e revisá-los fica para o ambiente publicado.

4.13 R17 - Dados pessoais reais em demonstrações (Alto → Médio)

- **Antes:** não havia como criar contas fictícias, e nas demonstrações alguém acabaria usando o próprio e-mail.
- **O que mudou:** dois comandos no backend. *npm run demo:accounts* cria contas fictícias (Jogador Demo 1, 2, 3...) num domínio reservado que nunca pertence a uma pessoa (*demo.smash.test*), com senhas aleatórias mostradas uma única vez. *npm run demo:clean* apaga todas essas contas, junto com as sessões, o progresso e a ofensiva.

  O cadastro continua pedindo só nome, e-mail e senha, o mínimo para a conta funcionar.
- **Como testamos:** duas contas foram criadas e depois apagadas ("2 conta(s) de demonstração apagada(s).").
- **O que falta:** usar os comandos é um procedimento da equipe: criar as contas antes de cada evento e apagá-las depois.

4.14 R23 - A API não enviava cabeçalhos de segurança (novo → Muito Baixo)

- **Antes:** a nova varredura com o OWASP ZAP mostrou que a API não enviava o cabeçalho *X-Content-Type-Options*, revelava o servidor no cabeçalho *X-Powered-By*, permitia guardar em cache respostas com dados do jogador e não isolava a origem contra ataques do tipo Spectre.
- **O que mudou:** a API passou a usar o helmet, que envia os cabeçalhos de segurança e retira o *X-Powered-By*, e toda resposta sai com *Cache-Control: no-store*.
- **Como testamos:** o ZAP foi rodado de novo depois da correção: os quatro avisos sumiram. Sobraram um aviso sobre a versão do próprio ZAP e um informativo, que confirma que as respostas não vão para o cache (Quadro 6 e Anexo B.8).

4.15 R24 - A API aceitava o segredo JWT de exemplo (novo → Médio)

- **Antes:** a revisão do histórico mostrou que o *.env.example* traz um segredo JWT de exemplo, e nada impedia a API de subir com ele. Como esse texto é público, quem o conhece conseguiria forjar o token de qualquer jogador.
- **O que mudou:** a API não sobe se o segredo for o de exemplo ou tiver menos de 32 caracteres, e mostra como gerar um aleatório (*openssl rand -base64 48*). O README da instalação foi atualizado.
- **Como testamos:** a API recusou iniciar com o segredo de exemplo (Anexo B); três testes de unidade cobrem a regra.

5 EVIDÊNCIAS DE TESTES DE SEGURANÇA

Os ataques simulados contra a API e a parede estão no Anexo B.1: cada linha traz o risco, o que foi tentado e o resultado, e o script *npm run security:test* repete todos eles. Os testes da API foram feitos com o backend e o banco locais, e os da parede com o emulador, que segue as mesmas regras do firmware. Os quadros abaixo resumem as dependências, as ferramentas de busca de vulnerabilidades e os testes automatizados.

Quadro 5 - Verificação de dependências (npm audit --omit=dev).

Parte	Entrega 1 (12/09/2026)	Antes das correções (25/10/2026)	Depois das correções
Backend	9 alertas	11 (1 crítico, 5 altos e 5 moderados)	0
Aplicativo	18 moderados	16 (1 crítico, 8 altos e 7 moderados)	5 altos, só no @expo/cli, fora do aplicativo instalado

Quadro 6 - Nova rodada de busca de vulnerabilidades.

Ferramenta	O que procura	Resultado
gitleaks 8.28	Chaves, tokens e senhas em todos os commits do repositório	O gitleaks não detectou nada nos 82 commits: ele procura chaves e tokens com formato conhecido, e não uma senha comum como smash1234. Por isso o histórico também foi revisado à mão, e essa revisão achou o smash1234 (R03, já conhecido) e o segredo JWT de exemplo, que virou o R24
Semgrep 1.140	Falhas de segurança no código: injeção, JWT, segredos, C/C++	112 regras em 371 arquivos (backend, aplicativo, firmware e scripts): nenhum achado
OWASP ZAP 2.16, baseline	Falhas visíveis nas respostas de uma rota pública	Antes: 0 falhas e 4 avisos de cabeçalhos e cache, que viraram o R23. Depois da correção: 0 falhas e 65 verificações aprovadas; sobraram um aviso sobre a versão do ZAP e um informativo
OWASP ZAP 2.16, todas as rotas	Ataques ativos (injeção, XSS e outros) nas 9 rotas da API, autenticado com um token de teste	0 falhas e 114 verificações aprovadas; só o aviso sobre a versão do ZAP. As 1.174 tentativas nas rotas /me foram recusadas pela validação (400). Limitação: nas rotas de login, o limite de tentativas (R06) bloqueou a maior parte dos ataques (429), e a saída da conta ficou fora da varredura para não revogar o token dela

Quadro 7 - Testes automatizados no commit de referência.

Parte	Suítes	Testes	Resultado
Backend	15	86	Todos passam
Aplicativo	17	117	Todos passam
Script de segurança	-	21	Todos passam
Firmware	-	Compilação	Compila (73% da memória de programa)

6 RISCOS MATERIALIZADOS E NOVOS RISCOS

6.1 Riscos materializados

- **R03:** a senha antiga da parede foi publicada no repositório público antes da correção, e o histórico do Git não pode ser apagado. A resposta é tratar a senha como vazada e nunca mais usá-la (item 4.3).
- **R11:** entre a Entrega 1 e esta revisão surgiram novos alertas de dependências, incluindo um crítico no backend. Isso confirma que a verificação precisa ser repetida a cada entrega, e não feita uma única vez.
- **R15:** o risco original era não ter tempo de implementar as respostas até a Entrega 2. Elas foram implementadas, então ele foi reformulado para o que sobrou: concluir até a apresentação, de 17 a 24/11/2026, os passos que dependem da placa e do servidor.

6.2 Novos riscos

As correções criaram ou revelaram nove riscos novos: três vieram das próprias correções (R20 a R22), dois da nova rodada de busca de vulnerabilidades (R23 e R24), três de efeitos colaterais das correções e da autenticação (R25 a R27) e um da revisão dos requisitos de privacidade do enunciado (R28). O Quadro 8 os classifica com as mesmas escalas da Entrega 1, e o Quadro 9 traz a resposta de cada um.

Quadro 8 - Novos riscos identificados na Entrega 2.

ID	Risco	Categoria	Prob.	Impacto	Severidade
R20	O segredo de pareamento vai dentro do pacote do aplicativo (*EXPO_PUBLIC_DEVICE_TOKEN*), e quem tiver o arquivo do app consegue extraí-lo.	Técnico	Baixa	Médio	Baixo
R21	Versões antigas do aplicativo deixam de salvar sessões, porque o servidor passou a exigir o ticket (R04).	Técnico	Média	Baixo	Médio
R22	Atrás de um proxy HTTPS, a limitação de tentativas veria todos os aparelhos com o mesmo IP e bloquearia todos juntos.	Técnico	Média	Médio	Médio
R23	A API não enviava cabeçalhos de segurança e permitia guardar respostas em cache (achado do ZAP).	Técnico	Muito Baixa	Baixo	Muito Baixo
R24	A API subia com o segredo JWT de exemplo do repositório, o que permitiria forjar tokens (achado na revisão do histórico).	Técnico	Muito Baixa	Muito Alto	Médio

ID	Risco	Categoria	Prob.	Impacto	Severidade
R25	Quem souber o e-mail de outra pessoa consegue bloquear o login dela por 15 minutos errando a senha de propósito (efeito do R06).	Técnico	Média	Baixo	Médio
R26	O segredo de pareamento vai no endereço da conexão (*?token=*), em texto claro, e pode aparecer em registros de rede.	Técnico	Baixa	Médio	Baixo
R27	O cadastro responde que o e-mail já está em uso, o que revela quem tem conta no Smash.	Técnico	Média	Baixo	Médio
R28	O jogador não consegue apagar a própria conta e seus dados pelo aplicativo, um direito do titular na LGPD.	Organizacional	Média	Médio	Médio

Quadro 9 - Respostas aos novos riscos.

ID	Estratégia	Ação	Responsável	Status
R20	Aceitar (ativo)	Segredo diferente para cada parede, trocado antes de cada apresentação. Ele só serve dentro da rede WPA2 da parede.	Firmware e Aplicativo	Em andamento
R21	Mitigar	Usar só a versão atual do aplicativo nas demonstrações. O servidor responde com uma mensagem clara: "A sessão precisa ser iniciada antes de ser registrada."	Aplicativo	Concluído
R22	Prevenir	Variável *TRUST_PROXY=true* no ambiente publicado, para o IP de cada aparelho vir do proxy. Já implementada e documentada no *.env.example*.	Backend	Concluído
R23	Mitigar	Cabeçalhos de segurança com o helmet e *Cache-Control: no-store* em todas as respostas (item 4.14).	Backend	Concluído
R24	Prevenir	A API não inicia com o segredo de exemplo nem com um segredo curto (item 4.15).	Backend	Concluído
R25	Aceitar (ativo)	É o custo conhecido de limitar tentativas por conta. O bloqueio é curto (15 minutos) e o limite por endereço impede que alguém bloqueie muitas contas de uma vez.	Backend	Aceito
R26	Aceitar (ativo)	A parede não registra o endereço pedido e só é alcançável dentro da própria rede WPA2. Trocar o segredo junto com a senha do Wi-Fi.	Firmware	Aceito
R27	Aceitar (ativo)	O aviso ajuda quem já tem conta, e o limite de tentativas do R06 impede testar muitos e-mails em pouco tempo. O login não revela se o e-mail existe: a mensagem de erro e o bloqueio por excesso de tentativas são iguais para e-mail com e sem conta (Anexo B.1).	Backend	Aceito
R28	Mitigar	Criar a exclusão da conta pelo próprio jogador. Até lá, a equipe apaga os dados a pedido.	Backend e Aplicativo	Não iniciado

7 COBERTURA DOS REQUISITOS DE SEGURANÇA

A seção 5 do enunciado do Projeto Interdisciplinar pede proteção contra o OWASP Top 10, cibersegurança do dispositivo IoT, atualização controlada e privacidade conforme a LGPD. O Quadro 10 liga os riscos deste relatório às categorias do OWASP Top 10 (2021) e do OWASP IoT Top 10 (2018), para mostrar o que já é coberto e o que fica fora.

Quadro 10 - Riscos do Smash nas categorias OWASP.

Categoria OWASP	Riscos e tratamento no Smash
A01 Controle de acesso quebrado	R04, R05 e R10: sessão aberta pelo servidor, um app pareado e revogação de token
A02 Falhas de criptografia	R01 (HTTPS) e R24 (segredo JWT forte)
A03 Injeção	Entradas validadas com lista de campos permitidos e consultas parametrizadas; Semgrep sem achados
A04 Design inseguro	R04: o resultado deixou de ser só declarado pelo aplicativo
A05 Configuração insegura	R08, R09 e R23
A06 Componentes vulneráveis	R11
A07 Falhas de identificação e autenticação	R06, R10, R25 e R27
A08 Falhas de integridade de software e dados	R04 e atualização do firmware só por cabo (item 7.1)
A09 Falhas de registro e monitoramento	R12
A10 SSRF	Não se aplica: a API não busca endereços externos
I1 Senhas fracas, previsíveis ou fixas no código	R03 e R24
I2 Serviços de rede inseguros	R02 e R07
I3 Interfaces do ecossistema inseguras	R01, R04 e R08
I4 Falta de atualização segura	Atualização só por cabo USB (item 7.1)
I5 Componentes inseguros ou desatualizados	R11
I6 Proteção de privacidade insuficiente	R17 e R28 (item 7.2)
I7 Transferência e armazenamento inseguros	R01, R02 e R26
I8 Falta de gestão do dispositivo	R05 e R18
I9 Configurações padrão inseguras	R03, R09 e R24
I10 Falta de proteção física	Placa e porta USB acessíveis: aceito, porque a parede só funciona em ambiente supervisionado

7.1 Atualização controlada do firmware

O firmware só pode ser atualizado por cabo USB, com o computador ligado à placa. Não existe atualização pela rede (OTA), então ninguém consegue trocar o

firmware a distância. O controle é físico: só a equipe grava a placa, e o quadro *hello* de cada conexão informa a versão do protocolo, o que faz o aplicativo detectar e avisar quando a placa está com um firmware antigo ("Parede incompatível"). Uma atualização pela rede, com firmware assinado, fica fora do MVP.

7.2 Privacidade, LGPD e proteção de menores

O Smash coleta só o necessário para a conta funcionar: nome, e-mail, senha (guardada apenas como hash bcrypt) e o histórico de treinos. Não coleta idade, documento, localização nem imagem, e a parede não tem câmera. Nas apresentações, só são usadas contas fictícias (R17).

O MVP não tem perfis de crianças e não pede a idade do jogador. Se o Smash passar a atender menores, a LGPD (art. 14) exige o consentimento de um dos pais ou responsável, coleta mínima e acesso restrito, como o enunciado pede; isso fica fora do MVP e é condição para abrir o produto a esse público. O direito de o titular apagar os próprios dados ainda não tem tela no aplicativo e está registrado como R28.

8 RISCO RESIDUAL E PRÓXIMOS PASSOS

Depois dos passos que faltam, nenhum risco fica Extremo, e só o R14, que é físico, continua Alto. Antes da apresentação na Semana FECAP de Tecnologia (17 a 24/11/2026), falta:

- **Publicar a API com HTTPS (R01, R22):** servidor com certificado reconhecido, *TRUST_PROXY=true* se ele estiver atrás de um proxy, e *EXPO_PUBLIC_API_URL* com *https://* no aplicativo.
- **Gravar o firmware novo na placa (R02, R03, R05 e R07):** criar o *secrets.h* com senha do Wi-Fi e segredo de pareamento novos, colocar o mesmo segredo em *EXPO_PUBLIC_DEVICE_TOKEN* no aplicativo e rodar *npm run security:test* apontando para a placa, para repetir nela os testes da parede do Anexo B.1.
- **Preparar o banco do ambiente publicado (R09):** conta própria, senha forte e só as permissões que a aplicação usa.
- **Usar contas fictícias nas demonstrações (R17):** *npm run demo:accounts* antes do evento e *npm run demo:clean* depois.
- **Repetir o npm audit (R11)** antes da apresentação e registrar o resultado.
- **Continuar a validação física (R14)** com a parede montada: um alvo, dois alvos lado a lado e depois os seis.
- **Criar a exclusão da conta pelo jogador (R28).**

Os responsáveis seguem as trilhas do Quadro 2 da Entrega 1: Firmware, Backend e Aplicativo, com a revisão do registro conduzida pelo Gerente de Projeto.

APROVAÇÃO

Quadro 11 - Aprovação do relatório.

Nome	Cargo	Assinatura	Data
Saulo Pereira de Jesus	Gerente do Projeto	Saulo Pereira de Jesus	06/11/2026
Felipe Vallim Soares	Responsável - Firmware (ESP32)	Felipe Vallim Soares	06/11/2026
Guilhermy Mariano Lisboa Garcia	Responsável - Backend (API e banco)	Guilhermy Mariano Lisboa Garcia	06/11/2026
Gustavo Oliveira Demetrio	Responsável - Aplicativo Mobile	Gustavo Oliveira Demetrio	06/11/2026
Prof. Victor Bruno Alexander Rosetti	Patrocinador do Projeto		

ANEXO A - REGISTRO DE RISCOS ATUALIZADO

O Quadro 12 traz os 19 riscos da Entrega 1 e os nove novos, com a severidade antes e depois das correções. O Quadro 13 liga cada correção ao commit que a implementa e aos arquivos onde ela pode ser conferida. Todos os commits citados estão no repositório público.

Quadro 12 - Registro de riscos atualizado (severidade na Entrega 1 e severidade residual).

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R01	A API usa HTTP sem criptografia e o aplicativo envia o token JWT nesse canal.	Extremo	Concluído no código; falta publicar	Baixa	Muito Alto	Médio
R02	O WebSocket da parede usa ws:// e aceita qualquer cliente sem autenticação.	Extremo	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R03	A senha do SoftAP é previsível e está versionada em repositório público.	Extremo	Concluído no código; falta gravar a placa	Muito Baixa	Alto	Baixo
R04	O resultado da sessão é declarado pelo aplicativo e aceito sem prova de que a partida ocorreu.	Alto	Concluído	Média	Médio	Médio
R05	Qualquer cliente conectado pode trocar o identificador de sessão da parede.	Alto	Concluído no código; falta gravar a placa	Baixa	Médio	Baixo
R06	Login e cadastro não têm limitação de tentativas.	Alto	Concluído	Baixa	Médio	Baixo
R07	O ESP32 não limita clientes, tamanho de mensagens ou frequência de comandos.	Alto	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R08	A API aceita qualquer origem CORS e escuta em todas as interfaces.	Médio	Concluído	Muito Baixa	Médio	Baixo
R09	O banco usa usuário e senha padrão de desenvolvimento como reserva.	Alto	Concluído	Baixa	Muito Alto	Médio
R10	O JWT dura sete dias e não possui revogação no servidor.	Médio	Concluído	Baixa	Médio	Baixo
R11	As dependências possuem alertas conhecidos no npm audit.	Alto	Concluído	Baixa	Médio	Baixo
R12	Eventos de segurança têm pouca trilha de auditoria.	Médio	Concluído	Baixa	Baixo	Baixo
R13	A integração MQTT prevista não tem controles de segurança definidos.	Alto	Encerrado: MQTT fora do escopo	-	-	-

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R14	Isolamento mecânico insuficiente faz a parede registrar o alvo errado.	Alto	Em andamento	Média	Alto	Alto
R15	Tempo insuficiente para concluir os passos pendentes até a apresentação (17 a 24/11). Reformulado.	Alto	Em andamento	Baixa	Alto	Médio
R16	Alteração de critérios, datas ou formatos das entregas.	Médio	Em andamento	Média	Médio	Médio
R17	Dados pessoais reais usados em testes e demonstrações.	Alto	Concluído	Baixa	Alto	Médio
R18	Falha ou dano da única placa ESP32 disponível.	Médio	Em andamento	Baixa	Alto	Médio
R19	Rede Wi-Fi do local de demonstração indisponível ou instável.	Médio	Encerrado	Muito Baixa	Médio	Baixo
R20	Segredo de pareamento dentro do pacote do aplicativo.	-	Em andamento	Baixa	Médio	Baixo
R21	Versões antigas do aplicativo deixam de salvar sessões.	-	Concluído	Média	Baixo	Médio
R22	Limitação de tentativas atrás de um proxy HTTPS.	-	Concluído	Média	Médio	Médio
R23	API sem cabeçalhos de segurança e com respostas guardáveis em cache.	-	Concluído	Muito Baixa	Baixo	Muito Baixo
R24	API aceitava o segredo JWT de exemplo.	-	Concluído	Muito Baixa	Muito Alto	Médio
R25	Bloqueio de login de outra pessoa pelo limite de tentativas.	-	Aceito	Média	Baixo	Médio
R26	Segredo de pareamento no endereço da conexão, em texto claro.	-	Aceito	Baixa	Médio	Baixo
R27	Cadastro revela se o e-mail já tem conta.	-	Aceito	Média	Baixo	Médio
R28	Sem exclusão da conta pelo próprio jogador (LGPD).	-	Não iniciado	Média	Médio	Médio

Quadro 13 - Commit e arquivos de cada correção (caminhos relativos a src/Entrega1).

ID	Commit	Arquivos
R01	810a16b e c98a80b	*backend/src/server-config.ts:24-42*, *backend/src/main.ts:21-43* e *frontend/src/config/env.ts:30-35*
R02	3946d66	*backend/firmware/smash_wall/wall_link.cpp:36-60* e *96-107*; *frontend/src/device/adapters/websocket/wall-protocol.ts:53-72*
R03	3946d66 e 38529e3	*backend/firmware/smash_wall/secrets_example.h*, *backend/firmware/smash_wall/wall_link.cpp:9-12* e *backend/firmware/smash_wall/config.h:33*

ID	Commit	Arquivos
R04	3b5380b	*backend/src/application/use-cases/start-training-session.use-case.ts:34*, *backend/src/application/use-cases/record-training-session.use-case.ts:83-97*, *backend/src/domain/training-session/training-session.entity.ts:72-80*, *backend/src/infrastructure/database/migrations/1757310000000-AddSessionTicket.ts:15* e *frontend/src/gameplay/services/sessionService.ts:64*
R05	3946d66	*backend/firmware/smash_wall/wall_link.cpp:73-82* e *109-117*
R06	9a8417b	*backend/src/presentation/http/auth-throttling.ts:15-32* e *backend/src/presentation/http/auth.controller.ts:13*
R07	3946d66	*backend/firmware/smash_wall/config.h:36-50* e *backend/firmware/smash_wall/wall_link.cpp:84-95*, *119-143* e *220*
R08	810a16b	*backend/src/server-config.ts:24-42* e *backend/.env.example*
R09	da16065	*backend/src/infrastructure/database/database.config.ts:16-32*
R10	e8a8476	*backend/src/application/use-cases/verify-access-token.use-case.ts:14*, *backend/src/application/use-cases/sign-out.use-case.ts:7*, *backend/src/infrastructure/infrastructure.module.ts:82*, *backend/src/presentation/http/auth.controller.ts:33* e *frontend/src/store/authStore.ts:43*
R11	b59160e e 3ce31d4	*backend/package.json* (versão fixada do multer), *backend/package-lock.json* e *frontend/package-lock.json*
R12	9b5698f	*backend/src/presentation/http/security-audit.ts:35-99* e *backend/src/app.module.ts:28*
R13	cb9f2ac	MQTT retirado do escopo do projeto
R17	a4cfb7e	*backend/src/infrastructure/database/demo-accounts.ts:18* e *backend/package.json:14-15*
R21	3b5380b	*backend/src/presentation/http/dto/record-session.dto.ts*
R22	9a8417b	*backend/src/server-config.ts:40* e *backend/src/main.ts:33-35*
R23	4e308b6	*backend/src/main.ts:26-32*
R24	a27d541	*backend/src/infrastructure/security/jwt-secret.ts* e *backend/src/infrastructure/infrastructure.module.ts:81* e *89*
Script	15a85cb e 9d5e68e	*scripts/security-tests.mjs* (*npm run security:test*) e *scripts/zap-openapi.yaml*

ANEXO B - SAÍDAS DOS TESTES

As saídas abaixo foram copiadas do terminal nas execuções feitas para este relatório. Só foram retiradas as cores, os horários e as linhas repetidas. Os endereços são da máquina de testes (127.0.0.1), e qualquer pessoa pode repetir os testes com os comandos indicados.

B.1 Script de testes de segurança

Comando: *npm run security:test*, com a API e o emulador da parede no ar.

```
API http://127.0.0.1:3990 · parede ws://127.0.0.1:8282


[OK]     R06  7 logins com senha errada na mesma conta -> 401, 401, 401, 401, 401, 429, 429
[OK]     R27  6 logins num e-mail que não tem conta -> 401, 401, 401, 401, 401, 429 (igual a uma
conta real)
[OK]     R27  cadastrar um e-mail que já existe -> 409 "Este e-mail já está em uso." (risco
aceito)
[OK]     R10  usar o token depois de sair da conta -> antes 200, saída 204, depois 401
[OK]     R04  começar o nível 5 sem ter passado do 4 -> 400 "Sessão de treino inválida: nível
ainda bloqueado."
[OK]     R04  enviar um resultado legítimo -> 201, 120 XP
[OK]     R04  enviar o mesmo resultado de novo -> 409 "Esta sessão de treino já foi registrada."
[OK]     R04  enviar resultado sem comprovante -> 400 "A sessão precisa ser iniciada antes de ser
registrada."
[OK]     R04  declarar 1 hora de jogo logo depois de começar -> 400 "Sessão de treino inválida:
duração maior que o tempo desde o início."
[OK]     R04  trocar o nível no resultado -> 400 "Sessão de treino inválida: modo ou nível
diferente do iniciado."
[OK]     R04  declarar 999.999 pontos com 6 acertos -> 400 "Sessão de treino inválida: pontuação
acima do máximo possível."
[OK]     R04  usar o comprovante como token de acesso -> 401
[OK]     R08  chamar a API a partir de um site qualquer -> sem permissão
[OK]     R23  conferir os cabeçalhos de segurança -> nosniff=nosniff, x-powered-by=ausente,
cache=no-store
[OK]     R02  conectar à parede sem senha de pareamento -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha de pareamento errada -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha certa -> hello
[OK]     R05  conectar um segundo aplicativo com a senha certa -> "parede ocupada", fechada: true
[OK]     R07  mandar uma mensagem de 400 bytes -> "comando grande demais"
[OK]     R07  mandar 30 comandos de uma vez -> 20 aceitos, 10 recusados
[OK]     R05  parear outro aplicativo depois que o primeiro saiu -> hello


21 de 21 testes passaram.
```

B.2 HTTPS com certificado de teste (R01)

```
$ curl -sk https://127.0.0.1:3989/rankings/xp -o /dev/null -w "%{http_code}\n"
200
$ curl -s http://127.0.0.1:3989/rankings/xp
000
(exit 52)
```

O código 000 e a saída 52 do curl indicam que a API não respondeu por HTTP na porta que serve HTTPS.

B.3 Linha do registro de auditoria após um login errado (R12)

```
{"level":"warn","pid":106119,"timestamp":1792984442933,"message":
{"event":"auth.sign_in","outcome":"failure","status":401,"method":"POST","path":"/auth/sign-
in","ip":"127.0.0.1","email":"a***@demo.smash.test"},"context":"Security"}
```

B.4 Firmware compilado sem o secrets.h (R03)

```
$ arduino-cli compile --fqbn esp32:esp32:esp32 smash_wall   (sem secrets.h)
smash_wall/wall_link.cpp:10:2: error: #error "Falta secrets.h: copie secrets_example.h para
secrets.h e troque as senhas."
   10 | #error "Falta secrets.h: copie secrets_example.h para secrets.h e troque as senhas."
```

B.5 API iniciada com o segredo JWT de exemplo (R24)

```
$ JWT_SECRET=troque-este-segredo-em-producao node dist/main.js
Error: JWT_SECRET fraco: use um valor aleatório com 32 caracteres ou mais (openssl rand -base64
48).
```

B.6 npm audit --omit=dev, antes e depois (R11)

```
$ cd backend && npm audit --omit=dev   # antes: commit 5444765
11 vulnerabilities (5 moderate, 5 high, 1 critical)
$ cd backend && npm audit --omit=dev   # depois: tag entrega2-ciberseguranca
found 0 vulnerabilities
$ cd frontend && npm audit --omit=dev   # antes: commit 5444765
16 vulnerabilities (7 moderate, 8 high, 1 critical)
$ cd frontend && npm audit --omit=dev   # depois: tag entrega2-ciberseguranca
node-forge  *
node-forge RSA PKCS#1 v1.5 signature verification accepts extra nested DigestAlgorithm elements -
https://github.com/advisories/GHSA-86w9-cpqp-85rv
zod  <4.6.6
5 high severity vulnerabilities
```

B.7 Testes automatizados (npm test)

```
$ npm --prefix backend test
Test Suites: 15 passed, 15 total
Tests:       86 passed, 86 total
$ npm --prefix frontend test
Test Suites: 17 passed, 17 total
Tests:       117 passed, 117 total
```

B.8 Busca de vulnerabilidades (Quadro 6)

```
$ docker run ghcr.io/gitleaks/gitleaks:v8.28.0 git /repo --redact
INF 82 commits scanned.
INF scanned ~1942510 bytes (1.94 MB) in 600ms
INF no leaks found
$ docker run semgrep/semgrep:1.140.0 semgrep scan --config p/typescript --config p/nodejs \
      --config p/javascript --config p/secrets --config p/jwt --config p/c ...
Ran 112 rules on 371 files: 0 findings.
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-baseline.py -t http://127.0.0.1:3992/rankings/xp
# antes da correção
WARN Insufficient Site Isolation Against Spectre Vulnerability [90004] risco=Low
WARN Server Leaks Information via "X-Powered-By" HTTP Response Header Field(s) [10037] risco=Low
WARN X-Content-Type-Options Header Missing [10021] risco=Low
WARN ZAP is Out of Date [10116] risco=Low
WARN Storable and Cacheable Content [10049] risco=Informational
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-baseline.py -t http://127.0.0.1:3992/rankings/xp
# depois da correção
WARN ZAP is Out of Date [10116] risco=Low
WARN Non-Storable Content [10049] risco=Informational
$ docker run ghcr.io/zaproxy/zaproxy:2.16.1 zap-api-scan.py -t zap-openapi.yaml -f openapi \
      -O http://127.0.0.1:3992 -z "<cabeçalho Authorization com token de teste>"   # todas as
rotas
Number of Imported URLs: 12
Total of 38 URLs
FAIL-NEW: 0    FAIL-INPROG: 0  WARN-NEW: 1     WARN-INPROG: 0  INFO: 0 IGNORE: 0       PASS: 114
WARN-NEW: ZAP is Out of Date [10116]
```